Optimizing Shipment Routes Under Cost, Time, and Risk Constraints

Technical report for the Aiqon AI Internship take-home. Repository: this project. All numbers below are produced by `shipment-decision evaluate` and `pytest`, and are reproducible from the literals in `src/shipment_decision/corpus.py`.

1. Problem framing and what "optimal" means here

The brief supplies, for each shipment, a small set of candidate routes with cost, transit time, delay probability and risk indicators, and asks for a system that evaluates them, recommends one according to defined criteria, and explains the choice to a logistics operator. The routes are given; nothing has to be discovered. I therefore treat this as **selection among supplied options**, not as path search or prediction. With three to five candidates, exact enumeration is optimal for any stated objective, and every term in the explanation is a number the operator can check.

"Optimal" is defined per shipment by a `priority` field with four meanings:

Priority	Definition
`min_cost`	lowest cost; ties by lower delay probability, then transit time, then route id
`min_time`	lowest transit time; ties by delay probability, then cost, then id
`min_risk`	lowest weighted risk score: delay probability 0.5, geopolitical risk 0.5 (delay alone if geopolitical is unavailable); ties by cost, time, id
`balanced`	lowest weighted score across cost, time, delay and geopolitical risk, with explicit weights or equal defaults

Hard limits (`max_cost`, `max_transit_hours`, `max_delay_probability`) are optional. A route that violates one is infeasible and is removed before any scoring; it cannot win by being excellent elsewhere, and limits are never relaxed. If nothing is feasible the system says so.

2. Scoring framework

For the feasible set F and each available objective x, values are min-max normalised within F: best = 0, worst = 1, and 0 for all routes if the range is zero or F has one route. The balanced score is S = sum of w_x * x_norm over objectives with positive weight, with weights summing to 1. Scores within 1e-9 tie and fall through to a fixed tie-break, so the recommendation is a deterministic total order independent of input order.

Design decisions that keep the objectives from double counting:

- **Delay probability vs transit time.** Planned duration and the chance of missing it are different inputs and are scored separately. No delay duration or monetary delay cost is assumed, because the brief provides neither.
- **Reliability** (1 - p) is the same fact as delay probability and is not scored again.
- **Congestion** is the brief's own example of a cause of delay. Since delay probability is already an objective, congestion is stored for context and excluded from the score; two routes differing only in congestion score identically (S13).
- **Geopolitical risk** is scored as the one risk indicator that plausibly measures something other than lateness (seizure, loss, denial of passage). This is a stated assumption, not a finding (section 5).
- **Missing risk data.** Geopolitical values must be present on every route or on none. Present on all: scored. Absent on all: the objective is disabled for the shipment and default weights become 1/3 each; nothing is filled with 0. Present on some: the input is rejected, because routes cannot be compared on an indicator only some of them carry. A positive geopolitical weight with no data is also rejected rather than silently renormalised.

Explicit weights must sum to 1 within 1e-6 and are never rescaled; the output always carries the effective weights and their source. The output also reports a `score_margin` (runner-up S minus recommended S; null for the lexicographic modes, which have no scalar objective), an unweighted `balance_gap` B over all available objectives, and two weight-independent dominance flags: non-dominated on all available objectives, and non-dominated on the objectives the mode consults.

3. Explanations

The explainer reads the result object and nothing else: it never recomputes a ranking, and a test asserts that every number in the text equals a result field. For S03, the brief's own example shape, the balanced text reads in part:

> Weighted score 0.2083 versus 0.3155 for runner-up C (margin 0.1071). ... Compared with C: cost 1,200 vs 1,000 (higher); transit time 250 h vs 240 h (higher); delay probability 0.080 vs 0.400 (lower); geopolitical risk equal at 0.200. Single-objective references among feasible routes: cheapest C (1,000), fastest F (180 h), lowest delay R (0.080) ... Dominated on every available objective by another feasible route: D.

The same shipment under `min_cost` recommends C and states what it gives up (delay 0.400 vs 0.080). Edge cases produce explicit sentences: a single feasible route, identical values on an objective, unavailable or zero-weight objectives, excluded routes with the limit they failed, and ties with the order used to break them.

4. Evaluation

Synthetic data. Twenty shipments (S01-S20) and four invalid fixtures (X01-X04) are hand-designed literals, each with a stated purpose: a four-way conflict, a tiny delay gain against a large cost increase, the brief's example, the two-route normalisation distortion and the same pair plus a third route, a dominant route, zero-range objectives, cost ties, single and empty feasible sets, geopolitical present/absent/zero-weighted, a near-tie at the clipping boundary, unequal explicit weights, and limits applied before normalisation. They are not jittered copies: adding a constant to every cost leaves min-max ranks unchanged, so repetition would only imitate sample size. Every winner and every balanced score was derived by hand as an exact rational before implementation, and the tests check them to 1e-9.

Method. There is no hidden correct route. Each of the four methods is run on every shipment; for the chosen route I record the raw value, the absolute gap to the best feasible value, and the relative gap (null when the best is 0, excluded from means with n reported, never replaced by 0), plus S under the shipment's balanced weights, B, and dominance. Aggregates over the 19 shipments with a feasible route:

Method	cost gap	time gap (h)	delay gap	geo gap (n=18)	mean S	mean B	non-dominated
min_cost	0.0	28.9	0.1163	0.0944	0.1681	0.7618	1.000
min_time	242.1	0.0	0.0558	0.2833	0.2631	1.0543	1.000
min_risk	84.3	29.6	0.0237	0.0111	0.1658	0.6528	1.000
balanced	47.4	23.2	0.0547	0.0694	0.1300	0.6094	1.000

Reading this honestly: `balanced` minimises S on every shipment by construction, so its lowest mean S is a consistency check, not evidence of quality. It pays for balance: worse cost than `min_cost`, worse time than `min_time`, and worse delay and geopolitical gaps than `min_risk`. Its lower mean B is a measured outcome on this corpus, not a guarantee; S15 (weights 0.7/0.1/0.1/0.1) and S18 (geopolitical weight 0) are cases where the S-minimiser and the B-minimiser differ (C vs R, Q vs P). All methods recommended non-dominated routes on this corpus. All four invalid fixtures were rejected with the expected code.

Sensitivity (deterministic perturbations, not calibrated uncertainty).

- *Weights*: each active weight +/-0.10 with the others rescaled proportionally (non-negative, sum 1, zeros stay zero). 148 perturbations, 10 flips (6.8%), on S07, S08, S14, S19. Mean original margin on flipped

shipments 0.021 vs 0.163 on unflipped: flips happen where the decision was already close, and S19 flips from an exact tie.

- *Uniform delay shift* +/-0.05, clipped to [0,1], feasibility held fixed: 1 flip in 38 for `balanced` (S14, where clipping at 1.0 breaks the shift invariance), 0 elsewhere. Without clipping a uniform shift moves min and max together and leaves every normalised value, every score and every raw ordering unchanged; this is why route-specific perturbations are also run.
- *Route-specific delay shift*: 12/106 flips for `balanced`, 10/106 for `min_risk`, 4/106 for each lexicographic mode. The lexicographic flips are all on S13 and S17, whose routes are identical: perturbing one of two identical routes necessarily changes an id-broken tie. This is a property of ties, not of the scoring weights.

5. Limitations and assumptions

- **Normalisation is relative to the candidate set.** With two feasible routes, min-max records only which route is worse on each objective, not by how much: S04 prefers a route that is 1 unit cheaper and faster over one with 0.35 lower delay probability. Adding a third, non-winning route (S05) reverses that. Raw values are shown beside every score for this reason, and scores are not comparable across shipments.
- **Equal default weights are a convention**, not an elicited preference. The sensitivity table shows where a 0.10 change matters.
- **Geopolitical risk is assumed independent of delay probability.** If a real dataset's delay estimates already reflect geopolitical disruption, this rule counts it twice. Nothing in the assignment lets that dependence be measured; the assumption is stated rather than tested.
- **Congestion never changes a recommendation.** An operator who considers congestion independent of the supplied delay probability will not see that preference represented.
- **The corpus is designed, not sampled.** It can expose implementation errors and illustrate trade-offs; it supports no claim about real-world savings or typical performance.
- **The system does not construct routes, predict attributes, or model time windows, connections or delay duration.** The brief supplies the routes and their attributes; adding a model would have reconstructed the synthetic generator.

6. Engineering

Python 3.10+, standard library only (pytest for tests). Modules: `validate` (schema, common-support rule, weights), `score` (feasibility, normalisation, selection, dominance, margin), `explain` (text from the result only; the only place rounding happens), `corpus`, `evaluate`, `cli`. Tolerances are documented constants: score tie 1e-9 (also the test tolerance), weight sum 1e-6, leg sum 0.01. 131 tests pass; they cover every corpus expectation in all four modes, the null and tie rules, the hand-calculated sensitivity flips, and that explanations contain only numbers from the result object.